Neural Network Design and Application for Image Analysis

A. Blokland, T. Ghaneh, P. Rickert

November 7, 2025

1 Introduction

Deep learning plays a central role in modern computer vision and has shown strong results in recognizing and learning patterns within image data. Using the TensorFlow library and its Keras API, it has become relatively easy to design and train neural networks for a wide range of tasks. In this report, we explore two connected tasks that assess how such models can be applied and optimized in practice. The first focuses on hyperparameter tuning, while the second involves developing a specialized model for a more complex image recognition problem.

One of the main difficulties in deep learning is choosing among the many possible model structures and hyperparameter settings. Model performance depends strongly on factors such as the optimizer, activation functions, regularization, and network depth. In the first part, we investigate these choices systematically using two common architectures: a Multi-Layer Perceptron (MLP) and a Convolutional Neural Network (CNN). The experiments are carried out on the Fashion-MNIST dataset, and the best-performing models are then tested on the more complex CIFAR-10 dataset to see how well they generalize to new data.

The second part focuses on a "tell-the-time" task, where a CNN is trained to read the time from images of analog clocks. This task introduces additional challenges, such as variations in lighting, rotation, and viewing angle. A key question is how to represent the time labels in a way that fits the circular nature of a clock face. To evaluate model performance, we use a task-specific error measure based on the absolute difference in minutes between the predicted and actual time, accounting for wrap-around cases (e.g., 11:55 and 0:05 differ by 10 minutes). To tackle this problem, we implement and compare these modeling strategies:

- Multi-class Classification: Treating each time interval (e.g., 30-minute, 10-minute, or 1-minute increments) as a unique class.
- Regression: Predicting a single continuous value representing time.
- Multi-head Architecture: Designing a network with two separate output heads for predicting hours and minutes. This approach is explored in two variants: a dual-classification model

and a hybrid model combining classification for the hour with regression for the minute, including an analysis of different loss weights.

This report will document our experimental methodology, analyze the performance of each approach through quantitative results and learning curves, and conclude with a discussion of the key insights gained from the experiments.

2 Methods

This section describes the algorithms and experimental setups for both tasks. All models were implemented using Python with the TensorFlow and Keras libraries. The experiments were run in a clear, repeatable manner, with the code available in the attached Jupyter Notebook.

We chose hyperparameters using a mix of known best practices from sources like the Deep Learning Tuning Playbook [1] and hands-on trial and error for each task. The idea was to build intuition and get a feel for what works best.

Task 1: Hyperparameter Exploration on Fashion-MNIST and CIFAR-10

The first task focused on gaining practical experience by systematically tuning Multi-Layer Perceptrons (MLPs) and Convolutional Neural Networks (CNNs).

2.0.1 Datasets and Preprocessing

Two benchmark image classification datasets were used:

- Fashion-MNIST: 60,000 training and 10,000 testing 28x28 grayscale images across 10 classes. The training set was partitioned into a 54,000-sample training set and a 6,000-sample validation set.
- CIFAR-10: 50,000 training and 10,000 testing 32x32 RGB color images across 10 classes. A 45,000/5,000 training/validation split was created from the original training data.

A data preprocessing pipeline was implemented:

1. Reshaping: Images were flattened into 1D vectors for MLP models (784-d for Fashion-MNIST, 3072-d for CIFAR-10) or prepared as 4D tensors for CNN models (28x28x1 or 32x32x3).
2. Normalization: Pixel values were scaled from [0, 255] to a [0, 1] range by dividing by 255.0.
3. Label Encoding: Using keras.utils.to_categorical, class labels were one-hot encoded.

2.0.2 Network Architectures and Hyperparameter Tuning

To structure the experiments, flexible functions were created to systematically build various model architectures and hyperparameters for each model family. A series of experiments was then conducted on Fashion-MNIST to evaluate the impact of various hyperparameter settings.

1. Multi-Layer Perceptron (MLP): The base MLP consisted of an input layer, a variable number of hidden Dense layers with Dropout, and a 10-unit softmax output layer. Hyperparameters included:
    - Network Depth & Width: 1, 3, or 5 hidden layers with 128, 256, or 512 neurons each.
    - Regularization: Dropout rates of 0.0, 0.2, and 0.4; L1, L2, and L1L2 kernel regularizers.
    - Activation Functions: ReLU, tanh, sigmoid, and elu.
2. Convolutional Neural Network (CNN): The base CNN featured two convolutional blocks (each with Conv2D, BatchNormalization, MaxPooling2D, Dropout), followed by a Dense head and a softmax output layer. Hyperparameters explored were:
    - Convolutional Filters: 16, 32, 64, or 128 filters in the first layer.
    - Dense Layer Neurons: 64, 128, or 256 units.
    - Dropout Rate: 0.0, 0.25, and 0.5.

All models were trained for up to 100 epochs with a batch size of 128, using either Adam or RMSprop optimizers (learning rate 0.001) and categorical_crossentropy loss. EarlyStopping with a patience of 10 on the validation loss was used to prevent overfitting. The top three performing configurations for both MLP and CNN models were then retrained on CIFAR-10 to assess generalization.

Task 2: "Tell-the-Time" CNN

This task aimed at predicting the time from images of an analog clock by minimizing a common sense error metric.

2.0.3 Dataset and Evaluation Metric

The dataset contains 18,000 grayscale images of clocks (150x150 pixels) with corresponding hour and minute labels. It was shuffled and split into a training set (80%), a validation set (10%), and a test set (10%). The primary performance metric is the common sense error, which calculates the mean absolute difference in minutes between the predicted time (T_{pred}) and the true time (T_{true}), accounting for the 12-hour (720-minute) cycle. The error for a single sample is:

$$\text{Error} = \min(|T_{true} - T_{pred}|, 720 - |T_{true} - T_{pred}|)$$

2.0.4 Base CNN Architecture and Training

All models were built using the same CNN architecture. This setup consisted of four convolutional blocks with increasing filter sizes (32, 64, 128, 256), each followed by BatchNormalization, MaxPooling2D, and Dropout. Heavy L2 regularization was applied to all convolutional and dense layers. A GlobalAveragePooling2D layer connected the feature extractor to a dense layer before branching into specific output layers. All models were trained for up to 200 epochs (batch size 32) using the Adam optimizer (learning rate 0.0001). ReduceLROnPlateau and EarlyStopping (patience=20, start_from_epoch=10) callbacks were employed.

2.0.5 Label Representations and Model Outputs

Three different modeling approaches were implemented to represent the time labels:

1. N-Class Classification: Time was converted into a single integer representing a discrete class. We experimented with three granularities:
   - 24 Classes: 30-minute intervals.
   - 72 Classes: 10-minute intervals.
   - 720 Classes: 1-minute intervals. The output layer was Dense(N, activation = 'softmax'), trained with categorical_crossentropy loss.
2. Regression: Time was converted to a single continuous value (e.g., 3:30 became 3.5). The output layer was Dense(1, activation='linear'), trained with mean_squared_error (MSE) loss.
3. Multi-head Models: Using the Keras Functional API, we created models with a shared base and two output heads:
   - Classification + Classification: One head predicted the hour (12 classes, softmax), and the other predicted the minute (60 classes, softmax), both using categorical_crossentropy loss. We systematically explored the impact of loss_weights with ratios of [3:1, 2.5:1, 2:1, 1.5:1, 1:1] for the hour and minute heads.
   - Classification + Regression: The hour head was a 12-class classifier (softmax, categorical_crossentropy), while the minute head was a regression output (1 neuron, sigmoid activation) predicting a normalized minute value in [0, 1]. This model used a combination of categorical_crossentropy and MSE loss.

3 Results

This section presents the results of the experiments detailed in the Methods section. Performance was evaluated based on test accuracy for Task 1 and the "common sense error" metric on the test set for Task 2.

Task 1: Hyperparameter Exploration and Generalization

3.0.1 Systematic Experiments on Fashion-MNIST

A series of experiments were conducted on the Fashion-MNIST dataset to explore the effects of different hyperparameters on MLP and CNN models. The performance was primarily measured by test accuracy, with EarlyStopping guided by validation loss. MLP and CNN Performance: As expected, CNNs significantly outperformed MLPs, highlighting their architectural advantage for image data. The best CNN achieved a test accuracy of 92.59%, while the top-performing MLP reached 89.27%. Table 1 summarizes the top three configurations for each architecture.

Table 1: *Top 3 MLP and CNN Configurations on Fashion-MNIST, Ranked by Test Accuracy*

Model	**Rank**	**Architecture / Parameters**	**Test Accuracy (%)**
MLP	1	1 Hidden Layer, 256 Neurons, 'sigmoid' activation, Dropout 0.2	89.27
	2	1 Hidden Layer, 512 Neurons, 'relu' activation, Dropout 0.4	88.96
	3	1 Hidden Layer, 256 Neurons, 'relu' activation, Dropout 0.4	88.82
CNN	1	128 Filters, 128 Dense Neurons, Dropout 0.5	92.59
	2	64 Filters, 256 Dense Neurons, Dropout 0.5	92.19
	3	64 Filters, 256 Dense Neurons, Dropout 0.25	92.14

Key observations from this exploration include:

- Regularization: Higher dropout rates (0.4-0.5) were consistently beneficial for both architectures, especially in models with higher capacity (more neurons or filters).
- Optimizers: The Adam optimizer consistently outperformed RMSprop and SGD across nearly all comparable configurations.
- CNN Architecture: For CNNs, increasing the number of convolutional filters generally led to higher accuracy, provided that strong dropout was also applied to prevent overfitting.

3.0.2 Generalization to CIFAR-10

The top three MLP and CNN configurations from Fashion-MNIST were retrained from scratch on the more complex CIFAR-10 dataset. The results, shown in Table 2, reveal a significant performance drop for all models.

The performance degradation was particularly severe for the MLP models, which failed to surpass 50% accuracy on CIFAR-10. This highlights the MLP's limitation in processing the spatial information and color channels present in the new dataset. In contrast, the CNNs demonstrated

Table 2: *Performance of Top 3 Fashion-MNIST Models When Retrained on CIFAR-10*

Model Type	**Rank**	**Architecture / Parameters**	**CIFAR-10 Test Acc. (%)**
MLP	1	1 Hidden Layer, 256 Neurons 'sigmoid' activation, Dropout 0.2	49.85
	2	1 Hidden Layer, 512 Neurons 'relu' activation, Dropout 0.4	44.69
	3	1 Hidden Layer, 256 Neurons 'relu' activation, Dropout 0.4	41.31
CNN	1	128 Filters, 128 Dense Neurons Dropout 0.5	78.65
	2	64 Filters, 256 Dense Neurons Dropout 0.5	76.84
	3	64 Filters, 256 Dense Neurons Dropout 0.25	75.43

much better generalization, with the top-performing model achieving a respectable 78.65% accuracy. This underscores the architectural superiority of CNNs for image tasks, as their convolutional layers learn transferable spatial features.

Task 2: "Tell-the-Time" CNN Results

The various approaches to the clock-reading task were evaluated based on the "common sense error". Table 3 provides a comprehensive comparison of the final test performance for each modeling approach.

Table 3: *Comparison of Common Sense Error for All "Tell-the-Time" Models*

Model Approach	**Output/Loss Configuration**	**Loss Weight (H:M)**	**Common Sense Error (min)**
A) Classification	24 Classes (30-min intervals)	N/A	8.10
	72 Classes (10-min intervals)	N/A	3.72
	720 Classes (1-min intervals)	N/A	11.34
B) Regression	Single Continuous Output (MSE)	N/A	20.28
C) Multi-head	Hour (Class) + Minute (Class)	1.0 : 1.0	1.01
	Hour (Class) + Minute (Reg)	1.0 : 1.0	3.90
	Hour (Class) + Minute (Class)	3.0 : 1.0	2.10
	Hour (Class) + Minute (Class)	**2.5 : 1.0**	**0.86**
	Hour (Class) + Minute (Class)	2.0 : 1.0	1.02
	Hour (Class) + Minute (Class)	1.5 : 1.0	1.06

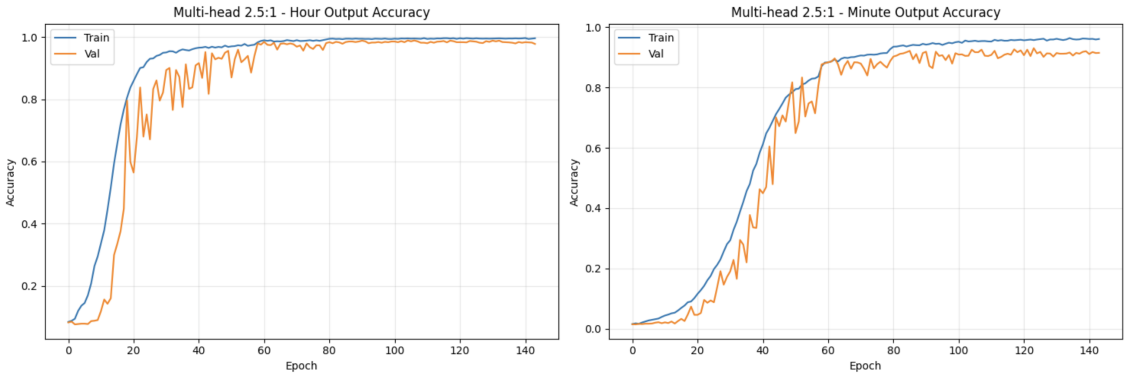


Figure 1: Training and validation learning curves for the best-performing multi-head model (2.5:1 loss weight ratio). Both accuracy (hour and minute) and loss metrics show smooth convergence and good generalization.

Analysis of Results:

- Classification Models: The performance of the single-output classification approach depended heavily on the label granularity. The 72-class model (10-minute intervals) performed best with an error of 3.72 minutes. Increasing the granularity to 720 classes was counterproductive (11.34 minutes), likely due to data sparsity across too many output classes.

- Regression Model: The direct regression approach performed the worst by a large margin, with an error of 20.28 minutes. This poor performance is attributed to the MSE loss function's inability to handle the cyclical nature of time, where it heavily penalizes predictions near the 12 o'clock boundary (e.g., predicting 0.1 for a target of 11.9).

- Multi-head Models: Splitting the problem into separate predictions for hours and minutes via a multi-head architecture proved to be the most effective strategy. The dual-classification head model (predicting hour and minute as separate classes) with equal loss weights achieved an error of just 1.01 minutes. The hybrid classification-regression model performed similarly to the 72-class model, indicating that classifying minutes is more effective than regressing them in this setup.

- Loss Weight Optimization: Fine-tuning the loss weights of the dual-classification multi-head model yielded the best overall performance. By assigning a higher weight to the hour prediction loss (a ratio of 2.5:1), we achieved a final common sense error of only 0.86 minutes. This suggests that encouraging the model to first correctly identify the hour provides a stronger feature basis for subsequently determining the precise minute.

The training history for this optimal model is shown in Figure 1. Both the hour and minute heads demonstrate stable convergence with a minimal gap between training and validation metrics, indicating that the model was well-regularized and generalized effectively.

4 Discussion and Conclusion

This assignment provided a practical investigation into the design, tuning, and application of neural networks for image-based tasks. Through a series of systematic experiments, we explored the impact of hyperparameter optimization and the effects of different architectural choices. This resulted in a highly accurate model for the "tell-the-time" problem.

4.1 Insights from Hyperparameter Tuning and Generalization

The experiments on Fashion-MNIST and CIFAR-10 confirmed several foundational principles of deep learning. The clear superiority of CNNs over MLPs on both datasets (92.6% vs. 89.3% accuracy on Fashion-MNIST) shows how important the model's built-in architectural design is. Convolutional layers in CNNs are built to take advantage of spatial structure and repeating patterns in images, something dense MLP layers cannot do.

The generalization test on CIFAR-10 was particularly interesting. The sharp drop in accuracy for all models highlighted that hyperparameters optimized for one dataset do not necessarily transfer directly to another, especially when the new dataset consists of more complex data. This emphasizes the "no free lunch" theorem [2] in a practical context: there is no universally optimal set of hyperparameters. The much larger performance drop observed in the MLP models (down to 49.9% accuracy) compared to the CNNs (down to 78.7%) further suggests that MLPs, without built-in spatial processing, handle changes in data more poorly.

4.2 The Critical Role of Label Representation

The "tell-the-time" task demonstrated that the representation of the target variable can be as important as the model architecture itself. The direct regression approach performed the worst (20.28 minutes of error), primarily because the Mean Squared Error loss function fails to account for the cyclical nature of time, heavily penalizing predictions near the 12 o'clock boundary (e.g., 11:59 vs. 0:01).

Formulating the problem as a classification task was more effective, though performance depended on granularity. The 72-class model (10-minute intervals) struck a good balance, achieving a low error of 3.72 minutes. However, increasing to 720 classes degraded performance substantially (11.34 minutes), likely due to data sparsity making it difficult for the model to differentiate between very similar adjacent classes.

The multi-head approach worked best because it split the task into two separate parts: one for hours and one for minutes. This approach is also intuitive, as the two clock hands look very different. The dual-classification version reached an error of only 1.01 minutes, and adjusting the loss weights to focus more on the hour prediction (about 2.5 times higher) improved the performance further to 0.86 minutes. This shows that getting the hour right first gives the network a solid base for fine-tuning the minute prediction.

4.3 Weaknesses and Future Work

A primary limitation of this work was the reliance on standard loss functions for a problem with cyclical labels. For the regression-based models, a custom loss function that explicitly models cyclicality (e.g., using sine/cosine transformations of the angle) could have significantly improved performance by eliminating the discontinuity at the 12 o'clock boundary. Additionally, our CNN architecture was relatively simple; more advanced designs like ResNet or the use of attention mechanisms [3] could potentially capture more complex features related to reflections and viewing angles. Finally, more sophisticated hyperparameter search methods, such as Bayesian optimization [1], could explore the parameter space more efficiently than our scripted grid search.

4.4 Conclusion

In summary, this assignment provided valuable hands-on experience in the practical application of deep learning. We demonstrated that while foundational models like MLPs are useful, specialized architectures such as CNNs are needed for more complex image tasks. The systematic tuning process showed the data-dependent nature of optimal hyperparameters and the difference that regularization can make. Most importantly, the "tell-the-time" task highlighted that the representation of the problem is as important as the model itself. By splitting a complex task into simpler, more manageable sub-problems, as demonstrated by our highly successful multi-head classification model, it is possible to optimize a neural network to learn more effectively and achieve better performance.

References

[1] V. Godbole, G.E. Dahl, Gilmer J., C.J. Shallue, and Z. Nado. Deep learning tuning playbook, 2023. Version 1.0.

[2] D.H. Wolpert and W.G. Macready. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997.

[3] C. Yang, W. Xie, and A. Zisserman. It's about time: Analog clock reading in the wild. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2508–2517, 2022.